TÀI LIỆU THIẾT KẾ

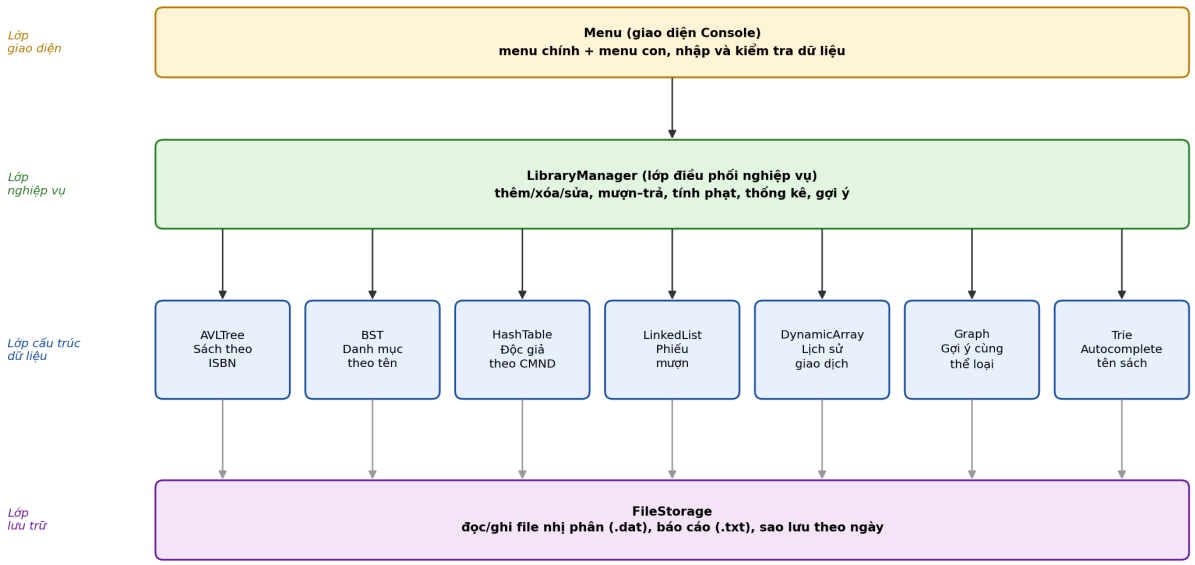
Chương trình Quản lý Thư viện bằng C++

Đồ án Tin học – Học kỳ I (2026-2027)

Giảng viên hướng dẫn	Lương An Vinh
Sinh viên thực hiện	Nguyễn Phúc Minh – MSSV: DH52301145
Tài liệu	Design (Tuần 2) – phiên bản 1.0

1. Kiến trúc tổng thể

Chương trình chia thành 4 lớp. Lớp giao diện (Menu) chỉ nhập/xuất và kiểm tra dữ liệu; lớp nghiệp vụ (LibraryManager) xử lý quy tắc thư viện; lớp cấu trúc dữ liệu chứa các CTDL tự cài đặt; lớp lưu trữ (FileStorage) đọc/ghi file. Cách chia này giúp mỗi file .h/.cpp có một nhiệm vụ rõ ràng và dễ kiểm thử riêng từng phần.



Hình 1. Kiến trúc các lớp / module của chương trình

2. Chọn cấu trúc dữ liệu cho từng chức năng

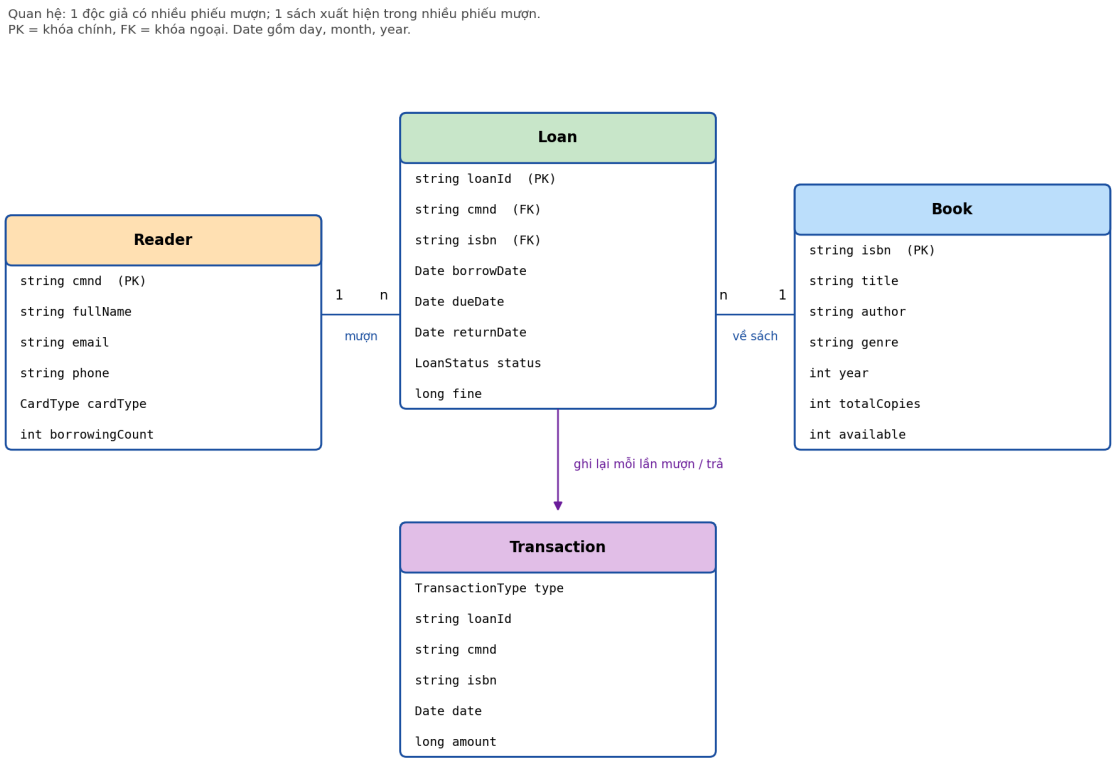
Bảng dưới cho biết mỗi CTDL dùng ở đâu, lý do chọn và độ phức tạp (n là số phần tử, L là độ dài chuỗi, V và E là số đỉnh và số cạnh).

#	CTDL	Dùng ở đâu	Vì sao chọn	Độ phức tạp
1	Cây AVL	Quản lý sách theo ISBN	Tự cân bằng nên tìm / thêm / xóa luôn nhanh, kể cả khi nhập ISBN theo thứ tự tăng dần.	Thêm, xóa, tìm: O(log n). Duyệt: O(n).
2	Cây BST	In danh mục sách theo tên (in-order)	Duyệt in-order cho thứ tự tên; đơn giản, dùng để so sánh hiệu năng với AVL.	Trung bình O(log n), xấu nhất O(n). In-order: O(n).
3	Bảng băm (separate chaining)	Quản lý độc giả theo CMND	Tìm theo mã chính xác, không cần thứ tự; xử lý va chạm bằng danh sách liên kết.	Trung bình O(1); xấu nhất O(n). Rehash khi hệ số tải > 0.75.

#	CTDL	Dùng ở đâu	Vì sao chọn	Độ phức tạp
4	Danh sách liên kết đơn	Danh sách phiếu mượn	Phiếu mượn liên tục thêm và đổi trạng thái, số lượng thay đổi nhiều.	Thêm đầu O(1). Tìm theo mã phiếu O(n).
5	Mảng động	Lịch sử giao dịch	Chỉ thêm vào cuối và duyệt tuần tự; truy cập theo chỉ số nhanh.	Thêm cuối O(1) khấu hao. Truy cập O(1).
6	Đồ thị (danh sách kề)	Gợi ý sách cùng thể loại	Mỗi đỉnh là một sách, cạnh nối các sách cùng thể loại; BFS/DFS tìm các sách liên quan.	BFS / DFS: O(V + E).
7	Cây Trie (tự tìm hiểu)	Tìm sách theo tiền tố tên (autocomplete)	Tìm theo tiền tố chỉ tốn thời gian theo độ dài tiền tố, không phụ thuộc số sách.	Thêm, tìm: O(L). Liệt kê kết quả: O(L + số kết quả).
8	File nhị phân	Lưu toàn bộ dữ liệu	Bản ghi độ dài cố định, đọc/ghi nhanh, không cần CSDL.	Đọc / ghi: O(n).

3. Định nghĩa các struct dữ liệu

Có 4 thực thể chính: **Book**, **Reader**, **Loan** (phiếu mượn) và **Transaction** (lịch sử giao dịch). Ngày tháng dùng struct **Date** riêng.



Hình 2. Các thực thể và quan hệ giữa chúng

```
struct Date { int day, month, year; };

struct Book {                    // khóa chính: isbn
    string isbn, title, author, genre;
    int year, totalCopies, available;
};

enum class CardType { Student, Teacher, Guest };
struct Reader {                  // khóa chính: cmnd
    string cmnd, fullName, email, phone;
    CardType cardType;
    int borrowingCount;          // sách đang mượn
};
```

```
enum class LoanStatus { Borrowing, Returned };
struct Loan {                       // khóa chính: loanId
    string loanId, cmnd, isbn;
    Date borrowDate, dueDate, returnDate;
    LoanStatus status;
    long fine;                      // phạt (VND)
};

enum class TransactionType { Borrow, Return };
struct Transaction {
    TransactionType type;
    string loanId, cmnd, isbn;
    Date date;
    long amount;                    // phạt phát sinh
};
```

4. Thiết kế các lớp chính

Lớp	Phương thức chính	Ghi chú
AVLTree	insert(Book), remove(isbn), find(isbn), forEach(visit), size()	Khóa so sánh là ISBN; có 4 phép xoay LL, RR, LR, RL.
BST	insert(title, isbn), inorder(visit)	Khóa là tên sách; chỉ lưu tên và ISBN, không lưu trùng toàn bộ sách.
HashTable	insert(Reader), remove(cmnd), find(cmnd), rehash()	Mảng các danh sách liên kết; hàm băm chuỗi CMND.
LoanList	addFront(Loan), findById(id), forEach(visit)	Danh sách liên kết đơn các phiếu mượn.
DynamicArray	push_back(Transaction), at(i), size()	Tự tăng gấp đôi dung lượng khi đầy.
Graph	addEdge(isbnA, isbnB), bfs(isbn, k), dfs(isbn)	Danh sách kề; bfs trả về tối đa k sách gần nhất.
Trie	insert(title, isbn), startsWith(prefix)	Chuẩn hóa tên về chữ thường, không dấu trước khi thêm / tìm.
FileStorage	saveAll(), loadAll(), backup(date)	Ghi / đọc 4 file .dat; tạo thư mục sao lưu theo ngày.
LibraryManager	addBook, borrowBook, returnBook, statistics, suggest	Điều phối các CTDL, kiểm tra quy tắc nghiệp vụ.

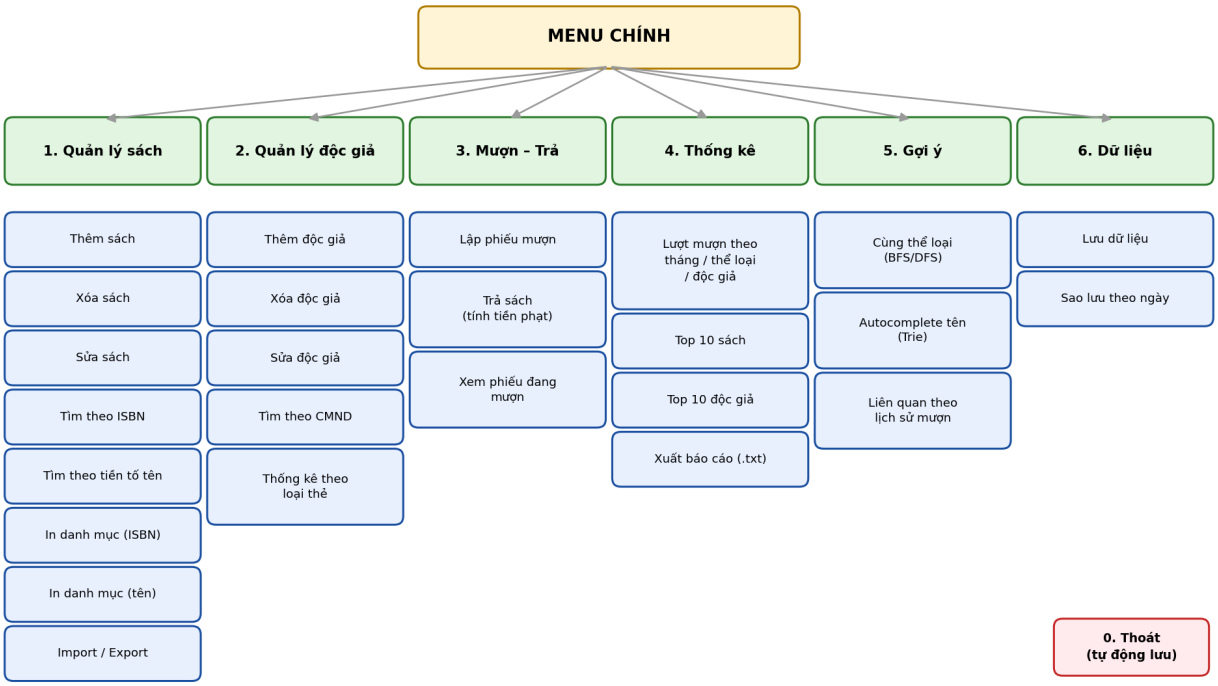
5. Thiết kế lưu trữ file

Mỗi file nhị phân gồm một phần đầu (header) rồi đến các bản ghi có độ dài cố định, nên đọc tuần tự rất nhanh. Chuỗi được lưu dạng mảng ký tự có độ dài tối đa.

File	Header	Bản ghi (độ dài trường)
books.dat	Mã nhận dạng "LBK1" (4 byte), số bản ghi (int)	isbn[20], title[100], author[60], genre[30], year, totalCopies, available (3 int)
readers.dat	"LRD1", số bản ghi	cmnd[16], fullName[60], email[60], phone[16], cardType, borrowingCount (2 int)
loans.dat	"LLN1", số bản ghi	loanId[16], cmnd[16], isbn[20], 3 Date (9 int), status (int), fine (long)
history.dat	"LHS1", số bản ghi	type (int), loanId[16], cmnd[16], isbn[20], Date (3 int), amount (long)

Khi khởi động, chương trình kiểm tra mã nhận dạng ở header; nếu file thiếu thì tạo dữ liệu rỗng, nếu header sai thì báo file hỏng và không ghi đè. Sao lưu theo ngày được chép vào thư mục `backup/YYYYMMDD/`. Báo cáo văn bản ghi vào `reports/`.

6. Thiết kế menu và luồng chương trình



Hình 3. Cây menu của chương trình

6.1. Luồng lập phiếu mượn

- Nhập CMND: tìm trong bảng băm; không thấy thì báo lỗi.
- Nhập ISBN: tìm trong cây AVL; không thấy thì báo lỗi.
- Kiểm tra ràng buộc: sách còn bản (available > 0), độc giả chưa đạt số sách tối đa theo loại thẻ, không có sách quá hạn.
- Tạo Loan (ngày hẹn trả = ngày mượn + 14 ngày), thêm vào danh sách liên kết; giảm available; tăng borrowingCount.
- Thêm một Transaction kiểu Borrow vào mảng động; lưu file.

6.2. Luồng trả sách

- Nhập mã phiếu: tìm trong danh sách liên kết; không thấy hoặc đã trả thì báo lỗi.
- Tính số ngày trễ = ngày trả − ngày hẹn trả. Nếu > 0 thì tiền phạt = số ngày trễ × 5.000 đồng.
- Cập nhật status = Returned, ghi returnDate và fine; tăng available; giảm borrowingCount.
- Thêm Transaction kiểu Return (kèm tiền phạt) vào mảng động; lưu file.

7. Xử lý lỗi và kiểm thử

- Kiểm tra dữ liệu nhập: không rỗng, đúng kiểu số, ngày hợp lệ; nhập sai thì yêu cầu nhập lại, không dừng chương trình.
- Trả về `bool` hoặc con trỏ `nullptr` cho các thao tác có thể thất bại (thêm trùng, tìm không thấy), thay vì kết thúc đột ngột.
- Mỗi CTDL có chương trình test riêng trong thư mục tests/ (AVL đã có, các CTDL khác bổ sung theo tuần), kiểm tra thêm / xóa / tìm / duyệt.
- Tuần 9 đo thời gian: AVL so với BST, bảng băm so với tìm tuần tự, Trie so với tìm chuỗi thông thường.

8. CTDL tự tìm hiểu: cây Trie

Trie là cây mà mỗi cạnh đại diện cho một ký tự; đường đi từ gốc đến một nút tạo thành một tiền tố. Mỗi nút giữ mảng con trỏ đến các nút con và một cờ đánh dấu kết thúc một tên sách (kèm ISBN). Tìm các tên bắt đầu bằng một tiền tố gồm hai bước: đi theo từng ký tự của tiền tố (O(L)), rồi duyệt cây con để liệt kê kết quả. So với việc duyệt toàn bộ n tên sách để so sánh tiền tố (O(n × L)), Trie nhanh hơn nhiều khi số sách lớn. Đổi lại Trie tốn nhiều bộ nhớ hơn do mỗi nút có mảng con trỏ. Phần demo và so sánh hiệu năng sẽ làm ở tuần 8 và 9.